

**SSN College of Engineering, Kalavakkam**  
**Department of Computer Science and Engineering**  
**V Semester - CSE 'A'**  
**UCS2511 NETWORKS LAB**  
**Exercise 9**  
**Address Resolution Protocol**

**Name :** Aathish Sivasubrahmanian 3122215001001

**Aim:**

Simulation of ARP using socket programming in C

Server.c

**server.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <netinet/in.h>
#include <unistd.h>

int main(int argc, char* argv[])
{

    int ser_fd, cl1, cl2;
    int clients[3];
    struct sockaddr_in ser_add, cl1_add, cl2_add, cl3_add;
    char ip[256];
    char reply[256];

    ser_add.sin_family=AF_INET;
    ser_add.sin_addr.s_addr=INADDR_ANY;
    ser_add.sin_port=htons(atoi(argv[1]));
```

```
if ((ser_fd=socket(AF_INET,SOCK_STREAM,0))<0)
{

    perror("\nSocket Failure");

exit(1); }

if ((bind(ser_fd,(struct sockaddr*)
&ser_add,sizeof(ser_add)))<0)

{
    perror("\nBinding Error");
    exit(1);

}

if ((listen(ser_fd,10))<0)
{

    perror("\nError");

exit(1); }

//cl1=accept(ser_fd,NULL,NULL)
/*c1=sizeof(cl1_add);
cl1=accept(ser_fd,(struct sockaddr*)&cl1_add,&c1);
```

```
int c2=sizeof(cl2_add);
cl2=accept(ser_fd,(struct sockaddr*)&cl2_add,&c2);
```

```
int c3=sizeof(cl3_add);
cl3=accept(ser_fd,(struct sockaddr*)&cl3_add,&c3);*/
```

```
for (int i=0;i<3;i++)
{
```

```
    clients[i]=accept(ser_fd,NULL,NULL);
}
```

```
//getting ip address and data
printf("Enter IP: ");
scanf("%s",ip);
```

```
/*write(cl1,ip,sizeof(ip));
write(cl2,ip,sizeof(ip));
write(cl3,ip,sizeof(ip));*/
for (int i=0;i<3;i++)
```

```
{
    write(clients[i],ip,sizeof(ip));
```

```
}
```

```
for (int i=0;i<3;i++)
{
```

```
read(clients[i],reply,sizeof(reply));
printf("%s\n",reply);
```

```
memset(reply,'\0',sizeof(reply));
```

```
}
```

```
return 0; }
```

### **client1.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <netinet/in.h>
#include <unistd.h>
```

```
int main(int argc, char* argv[])
{
```

```
    int cl_fd,n,c;
    struct sockaddr_in cl_add;
    char msg[256];
    char ip[256];
    char mac[256];
    strcpy(ip,argv[2]);
    strcpy(mac,argv[3]);
```

```
    cl_add.sin_family=AF_INET;
    cl_add.sin_addr.s_addr=INADDR_ANY;
```

```
    cl_add.sin_port=htons(atoi(argv[1]));
```

```

if ((cl_fd=socket(AF_INET,SOCK_STREAM,0))<0)
{

    perror("\nSocket Failure");

exit(1); }

connect(cl_fd,(struct sockaddr*) &cl_add,sizeof(cl_add));
printf("C1 connected to server");
//reading ip address
read(cl_fd,msg,sizeof(msg));

if (strcmp(ip,msg)==0)
{

    printf("\nSending MAC to server %s",mac);

    write(cl_fd,mac,sizeof(mac));
}

else {

    printf("\nIP not matching");
}

return 0; }

```

### **client2.c**

```

#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

```

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <netinet/in.h>
#include <unistd.h>
```

```
int main(int argc, char* argv[])
{
```

```
    int cl_fd,n,c;
    struct sockaddr_in cl_add;
    char msg[256];
    char ip[256];
    char mac[256];
    strcpy(ip,argv[2]);
    strcpy(mac,argv[3]);
```

```
    cl_add.sin_family=AF_INET;
    cl_add.sin_addr.s_addr=INADDR_ANY;
    cl_add.sin_port=htons(atoi(argv[1]));
```

```
    if ((cl_fd=socket(AF_INET,SOCK_STREAM,0))<0)
    {
```

```
        perror("\nSocket Failure");
```

```
    exit(1); }
```

```
    connect(cl_fd,(struct sockaddr*) &cl_add,sizeof(cl_add));
    printf("C2 connected to server");
    //reading ip address
    read(cl_fd,msg,sizeof(msg));
```

```
if (strcmp(ip,msg)==0)
{

    printf("\nSending MAC to server %s",mac);

    write(cl_fd,mac,sizeof(mac));
}

else {

    printf("IP not matching");
}

return 0; }
```

### **client3.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <netinet/in.h>
#include <unistd.h>

int main(int argc, char* argv[])
{

    int cl_fd,n,c;
    struct sockaddr_in cl_add;
    char msg[256];
    char ip[256];
    char mac[256];
```

```
strcpy(ip,argv[2]);
strcpy(mac,argv[3]);
```

```
cl_add.sin_family=AF_INET;
cl_add.sin_addr.s_addr=INADDR_ANY;
cl_add.sin_port=htons(atoi(argv[1]));
```

```
if ((cl_fd=socket(AF_INET,SOCK_STREAM,0))<0)
{
```

```
    perror("\nSocket Failure");
```

```
exit(1); }
```

```
connect(cl_fd,(struct sockaddr*) &cl_add,sizeof(cl_add));
printf("C3 connected to server");
//reading ip address
read(cl_fd,msg,sizeof(msg));
```

```
if (strcmp(ip,msg)==0)
```

```
{
    printf("\nSending MAC to server %s",mac);
    write(cl_fd,mac,sizeof(mac));
```

```
} else {
```

```
    printf("IP not matching");
}
```

```
return 0; }
```

O/P:

```
$/sarp 8080
Enter IP: 11.11.11.13
@
CC:CC:CC:CC
```

```
$ ./carp1 8080 11.11.11.11 AA:AA:AA:AA
C1 connected to server
IP not matching
```

```
$ ./carp3 8080 11.11.11.13 CC:CC:C:CC
C3 connected to server
Sending MAC to server CC:CC:CC:CC
```

```
$ ./carp2 8080 11.11.11.12 BB:BB:BB:BB
C2 connected to serverIP not matching
```

### Learning Outcomes:

I learned how to simulate ARP using socket programming.